Mayo Clinic

Digital Product Engineering – Test 2

Hameem Mahdi M.S.E., Ph.D.

Deterministic Personalization, Recommendation & Search Engine

March 3, 2026

Language: Java 8 | Build: Fat JAR (no Maven/Gradle required)

1. Project Overview

This project implements a fully deterministic personalized search and recommendation engine as required by the Mayo Clinic Digital Product Engineering Test 2 specification. The engine processes four test datasets of health content, user profiles, queries, and scoring rules — producing ranked results and personalized recommendations for each query.

Key capabilities:

- Indexes health content items (articles, guides)
- Runs keyword-based search against each user query
- Applies personalization boosts based on user specialty and channel preferences
- Produces deterministically ranked top-K results
- Generates personalized recommendations excluding already-viewed content
- Produces identical output for identical inputs (100% deterministic)

2. Project Structure

Path	Description
src/main/java/.../Main.java	CLI entry point – reads input files, runs engine, writes output
src/main/java/.../engine/SearchEngine.java	Core scoring, ranking, tie-break, and recommendation logic
src/main/java/.../model/Content.java	POJO for a content item (article/video)
src/main/java/.../model/User.java	POJO for a user profile
src/main/java/.../model/Query.java	POJO for a search query
src/main/java/.../model/Rules.java	POJO for scoring rules and weights
src/main/java/.../model/Weights.java	POJO for individual weight values
src/main/java/.../output/Output.java	Top-level output wrapper
src/main/java/.../output/Result.java	Ranked result per query
src/main/java/.../output/Recommendation.java	Recommendations per query
src/main/java/.../output/Metrics.java	Run metrics (queriesProcessed, topK, recCount)
lib/	Jackson 2.15.2 JARs (databind, core, annotations)
outputs/	Generated output files – the graded deliverables
search-engine.jar	Pre-built fat JAR (ready to run, no build needed)
build.sh / build.bat	Build scripts (Linux/Mac and Windows)
run_all.sh	Runs all 4 datasets with one command

3. Scoring Algorithm

For each query, every content item is scored using the following deterministic formula:

#	Factor	Formula	Weight (rules.json default)
1	Keyword Match	+keywordMatch per query token found in title	50
2	Tag Match	+tagMatch per query token matching a content tag	30
3	Specialty Boost	+specialtyBoost if content.specialty ∈ user.preferredSpecialties	20
4	Channel Boost	+channelBoost if content.channel = user.channelPreference	10
5	Popularity	+popularityScore × popularityWeight	×0.5
6	Freshness	+(1 / freshnessDays) × freshnessWeight × 100	×0.3

3.1 Tie-Breaking Rules (Mandatory Order)

When two content items have an identical score, the following rules are applied in strict order:

- 1st – Lower freshnessDays (newer content wins, ascending)
- 2nd – Higher popularityScore (more popular content wins, descending)
- 3rd – Lexicographically smaller contentId (alphabetical, ascending)

This guarantees a complete and stable ordering with no ambiguity.

4. Recommendation Logic

After ranking, recommendations are generated per query using the following logic:

Step 1 – Build Exclusion Set

- All content IDs returned in the top-K ranked results
- All content IDs in user.recentContentIds

Step 2 – Score Remaining Items

- +tagMatch for each tag shared with the #1 ranked result
- +specialtyBoost if content.specialty ∈ user.preferredSpecialties
- +popularityScore × popularityWeight
- +(1 / freshnessDays) × freshnessWeight × 100

Step 3 – Sort and Return

Apply the same tie-break rules. Return exactly rules.recommendationCount items.

5. How To Run

5.1 Prerequisites

Requirement	Version	Notes
Java (JDK or JRE)	8+	Already included in PATH on test machine
python-docx	any	Only needed to regenerate this document
Maven / Gradle	N/A	NOT required – fat JAR approach used

5.2 Navigate to the search-engine folder

```
cd "C:/Users/m141014/OneDrive - Mayo Clinic/Desktop/applications/Digital
Product Engineering Roles/Coding Sessions/TBD/TEST 2/search-engine"
```

5.3 Run All Datasets (Recommended)

```
bash run_all.sh
```

5.4 Manual CLI – Individual Datasets

```
java -jar search-engine.jar --case
"../engineering_test_2/Archive/cases/dataset_1" --out
"outputs/dataset_1.output.json"

java -jar search-engine.jar --case
"../engineering_test_2/Archive/cases/dataset_2" --out
"outputs/dataset_2.output.json"

java -jar search-engine.jar --case
"../engineering_test_2/Archive/cases/dataset_3" --out
"outputs/dataset_3.output.json"

java -jar search-engine.jar --case
"../engineering_test_2/Archive/cases/dataset_4" --out
"outputs/dataset_4.output.json"
```

5.5 Build from Source (Optional)

```
bash build.sh
```

This recompiles all Java sources and repackages search-engine.jar. Only needed if source files are modified.

6. Output Format

Each dataset produces one JSON file with exactly three top-level keys:

```
{
  "results": [
    { "queryId": "Q1", "rankedContentIds": ["C8","C6","C1","C9","C4"] }
  ],
  "recommendations": [
    { "queryId": "Q1", "recommendedContentIds": ["C7","C2","C10"] }
  ],
  "metrics": {
    "queriesProcessed": 2,
    "topK": 5,
    "recommendationCount": 3
  }
}
```

7. End-to-End Test Results

All evaluation criteria were verified by running each dataset twice and programmatically validating the outputs:

Evaluation Criterion	dataset_1	dataset_2	dataset_3	dataset_4
Correct output format (3 required keys)	PASS	PASS	PASS	PASS
Query count matches input	PASS (2/2)	PASS (5/5)	PASS (10/10)	PASS (15/15)
TopK ≤ 5 per result	PASS	PASS	PASS	PASS
Recommendation count ≤ 3	PASS	PASS	PASS	PASS
No ranked/rec overlap	PASS	PASS	PASS	PASS
No recommendations from recent content	PASS	PASS	PASS	PASS
Metrics values correct	PASS	PASS	PASS	PASS
No duplicate contentIds in results	PASS	PASS	PASS	PASS
Stability (Run 1 == Run 2)	PASS	PASS	PASS	PASS

8. AI Usage

Task	AI Involvement
Spec extraction from PDF	Used pdftotext + Copilot to parse and summarize requirements
Architecture design	Copilot proposed Java package layout and fat-JAR approach
All Java source code	Copilot generated all 11 source files
Build scripts	Copilot generated build.sh, build.bat, run_all.sh
Path / Windows issue fix	Copilot diagnosed OneDrive spaces issue, created NTFS junctions
Validation / testing	Copilot wrote Python validation script and ran determinism checks
Documentation	Copilot wrote README, RUN_RESULTS, AI_USAGE_LOG, this document

Tool: GitHub Copilot (Claude Sonnet 4.6) via VS Code

Human decisions: Java language choice, admin credentials for NTFS junctions, final review.